RobinHood AI

Privacy-Preserving Tax Intelligence System

Project Report & Technical Working Guide

Flask · React + Shadcn · Supabase · Fernet · IsolationForest

Assessment Year Context: FY 2024-25 / AY 2025-26 (India, Dual Regime)

1. What the Project Does

RobinHood AI is a full-stack tax compliance platform for India's dual income-tax regimes. Citizens get a live optimizer that compares the New vs Old regime for their income and deductions and tells them the exact income at which the better choice flips. Administrators get a secure command center with an encrypted taxpayer registry, a machine-learning risk matrix, a compliance-notice workflow, and a tamper-evident audit ledger that proves no record was ever silently edited.

Key capabilities

Area	Capability
Intelligence	Statutory slab math with 4% cess, surcharge tiers (10/15/25/30%), Section 87A rebates, Chapter VI-A caps; marginal-swap threshold found by bisection; IsolationForest anomaly screening mapped to a 0-100 risk index.
Privacy	Names, PANs and asset values stored as Fernet ciphertext with a distinct HKDF-derived key per column; PAN lookups via HMAC blind index - the database never stores searchable PII.
Integrity	Audit ledger where each row's hash covers the previous row's hash; DB triggers enforce append-only writes; verification pinpoints the exact row where tampering occurred.
Access control	Hybrid auth: Supabase JWT for citizens, PBKDF2-signed Flask sessions for admins (rate-limited), real per-role Row Level Security policies.
Resilience	Offline demo mode mirrors the Supabase API in memory, so the full pipeline is demonstrable without network.

2. Architecture - Three Tiers

Tier	Technology	Responsibility
Visual Layer	React 18, Vite, TypeScript, Tailwind, Shadcn UI, TanStack Query, Recharts	Citizen optimizer (sliders + crossover chart), admin command center (stats, registry, risk matrix, ledger).
Intelligent Backend	Flask 3 gateway + SovereignTaxAgent (scikit-learn)	Regime optimization, risk scoring, compliance evaluation, notice generation, auth enforcement, audit writes.
Secure Data Layer	Supabase (PostgreSQL via PostgREST)	Field-level encrypted storage, blind-index search, chained append-only ledger, RLS.

Request flow (citizen assessment)

POST /api/tax/assess (Bearer JWT) → input validation → optimize_regime_matrix() → IsolationForest risk score → PII encrypted per field → row inserted into Supabase **taxpayers** → chained event appended to **system_audit_ledger** → 201 with recommendation, savings, risk index and human-readable reasons.

Backend modules

File	Role
app.py	Flask gateway: routes, CORS, rate limiting, safe error handling, SPA serving.
tax_agent.py	SovereignTaxAgent: both regime engines, swap threshold, ML risk engine, notice content.
crypto.py	Per-field Fernet encryption (HKDF) + HMAC blind index for PAN.
audit.py	record_event / verify_chain / get_trail for the chained ledger.
db.py	Supabase client singleton, error normalization, in-memory offline fallback store.
models.py	Taxpayer/Notice ORM-style layer: encrypt on write, decrypt on read.
auth_decorators.py / admin_auth.py	@require_supabase_auth, @require_role, PBKDF2 admin login/logout/me.
validation.py	Required fields, numeric ranges, length caps - clean 400 responses.
robinhood_cli.py	Admin CLI: optimize, risk, audit verify/trail, batch notices, model retrain, login-hash.
supabase_schema.sql	Tables, RLS policies and append-only triggers to run once in Supabase.

3. How It Works - The Tax Engine (FY 2024-25)

Component	New Regime	Old Regime
Slabs	Nil up to 3L; 5/10/15/20% to 15L; 30% above	Nil up to 2.5L; 5% to 5L; 20% to 10L; 30% above (rebate-based relief)
Standard deduction	75,000	50,000
Deductions allowed	None (salaried)	80C up to 1.5L, 80D up to 25k/50k senior, HRA, 24(b) up to 2L
Section 87A rebate	Full nil up to 7L taxable	12,500 up to 5L taxable
Surcharge	10/15/25% tiers (capped at 30%)	10/15/25/30% above 50L/1Cr/2Cr/5Cr
Cess	4% Health & Education	4% Health & Education

Order of computation follows statute: deductions → slab tax → 87A rebate → surcharge → cess. Worked example: income 15,00,000 with 80C 1.5L and 80D 50k gives New = 1,40,400 vs Old = 2,02,800 → recommendation NEW, savings 62,400.

The marginal-swap threshold

Holding deductions fixed, the engine scans the income range for the point where old_tax - new_tax strictly changes sign, then bisects it to rupee precision. That single number - drawn as the crossover marker on the portal chart - tells a citizen exactly when their optimal regime flips. If one regime dominates everywhere for a profile, the engine honestly returns no threshold.

ML risk screening

An IsolationForest (contamination 0.04, fixed seed) is trained on a synthetic lognormal taxpayer population blended with live rows; score_samples is linearly mapped to a documented 0-100 index. It is labelled anomaly screening, not fraud detection, and every flagged score comes with concrete reason strings (e.g. 'paid only 30% of expected tax').

4. How It Works - Security Model

Field-level encryption with per-column keys

One master Fernet key lives only in the server .env. Each column gets its own derived key: key(field) = HKDF-SHA256(master, info=field_name). Decrypting one leaked column key still cannot read the others. Because Fernet ciphertext is randomized, searchability is restored with a blind index: pan_index = HMAC-SHA256(pepper, normalized PAN) - deterministic, yet non-reversible.

Tamper-evident chained audit ledger

Each ledger row stores prev_hash and payload_hash = SHA256(prev_hash | timestamp | action | actor | target | payload). Genesis starts from 64 zero hex chars. Editing or deleting any historical row breaks every subsequent hash, so verify_chain() reports the exact log_id where tampering began. There is deliberately no foreign key on the audited user, so deleting a taxpayer can never erase the trail, and DB triggers block UPDATE/DELETE on the ledger.

Authentication & authorization

Surface	Mechanism	Enforcement
Citizen / machine API	Supabase JWT via Authorization: Bearer	Validated server-side; 401 otherwise
Admin dashboard & CLI	Flask signed session cookie (PBKDF2-verified login, 5 attempts/min)	@require_role('admin') on every admin route
Database	Supabase Row Level Security	Citizens select only their own taxpayer row; ledger and notices are service-role only
Gateway hardening	CORS pinned to one origin, rate limiting, input validation	No internal error strings ever reach clients

Disclosed legacy risk: an early revision committed a MySQL root password in source. It is treated as public; the credential was eliminated by the full migration to Supabase and secrets now come from a gitignored .env.

5. The Application Screens

Route	Audience	What happens
/	Citizen	Dual-Regime Optimizer: income / 80C / 80D sliders feed a debounced API call; recommendation card shows both regime liabilities and savings; the crossover curve marks the flip income; 'File Secure Assessment' encrypts PII, scores risk and audit-logs the event.
/login	Admin	Rate-limited PBKDF2 login establishing a signed HttpOnly session; the attempt itself is recorded in the ledger.
/admin	Admin	Command center: compliance stat cards, ML Risk Matrix scatter (income vs paid ratio, sized/colored by risk), encrypted Taxpayer Registry with flag/notice/payment actions, Audit Ledger with one-click Verify Integrity, live risk insights and collection trends.

6. Running the System

Local / offline demo

Step	Command
1. Install backend deps	pip install -r requirements.txt
2. (optional) configure Supabase	copy .env.example to .env, set URL, service-role key, CRYPTO_KEY, ADMIN_PASSWORD_HASH
3. Apply schema (real project)	run supabase_schema.sql in the Supabase SQL Editor, then python setup_db.py
4. Build frontend	cd frontend && npm install && npm run build
5. Run	python app.py (serves API + SPA at http://localhost:5000)
Demo admin login	admin@robinhood.gov.in / admin-demo-123 (offline fallback only)

Quality gates

A 30-test pytest suite covers tax edge cases (the 7L and 5L rebate boundaries, surcharge crossovers at 50L/1Cr/2Cr/5Cr), encryption round-trips, blind-index determinism, ledger tamper detection, JWT/session 401/403 paths and the login rate limit. The full browser flow - optimize, file assessment, admin login, verify chain - is exercised end to end; tamper demo: hand-edit one ledger row in the table editor, press Verify Integrity, and the exact broken log_id is reported.

API surface (summary)

Endpoint	Auth	Purpose
/api/tax/optimize, /api/tax/curve	public	Regime matrix + crossover curve for the sliders
/api/tax/assess	Supabase JWT	Full gateway: optimize, risk, encrypt, persist, audit

Endpoint	Auth	Purpose
/api/admin/login, /api/auth/me	rate-limited / session	Admin hybrid auth
/api/users, /api/stats, /api/risk, /api/trends	admin	Registry, aggregates, live ML insights
/api/notices/*, /api/payments/process	admin	Persisted tailored notices, payments
/api/audit/verify, /api/audit/trail	admin	Chain integrity check and ledger view

RobinHood AI - Project Report · Built with Flask, React, Supabase, scikit-learn and cryptography.